When does retrieval stop helping?

A controlled comparison of seven long-context strategies

under a fixed dollar budget

Gursahib Singh
gursahib99888@gmail.com

May 2026

Abstract

Large language models are now smart enough to read entire books at once, yet most production systems still chop documents into pieces and retrieve a few before answering — a technique called RAG (Retrieval-Augmented Generation). With dozens of RAG variants and several non-retrieval alternatives, practitioners face a real question: *which one should I actually use?* We answer this for one concrete setting: a small but carefully designed test corpus (about 8,000 words), a hand-authored set of 34 questions probing five different abilities, a single language model (`gemini-3-flash-preview`), and a hard project budget of $5. Under these matched conditions we run four RAG variants and three non-RAG long-context techniques, each across three random seeds, and report token F1 along with bootstrap 95% confidence intervals, latency, and dollar cost. Three findings stand out: (1) when the corpus comfortably fits in a single prompt, the two non-retrieval approaches *outperform* every RAG variant; (2) pre-summarizing the corpus catastrophically destroys factoid recall (F1 $\approx$ 0.21), even when the summarizer is explicitly told to preserve facts; and (3) the strict-refusal prompt is universally honest — six of seven approaches refuse correctly on every probe whose answer is not in the corpus. We close with a one-page recommendation chart showing which approach to pick depending on whether your priority is cost, quality, or honesty.

In plain English

The question this paper answers — without the jargon — is: **when should you give a language model the whole document, and when should you let it look up snippets?** The answer depends on how big the document is, how much you can spend, and how much you mind the model occasionally inventing things. Our experiment with seven different setups suggests a simple rule: if the document fits, hand it over. If it doesn't, a small, focused look-up is fine. But never summarize first — you'll lose exactly the facts you came for.

1 Why this matters

Modern AI assistants — the kind people use to draft emails, summarize PDFs, or query company wikis — almost always face the same problem. The user has a question. The answer is somewhere in a pile of documents. The assistant has a finite "attention span" (the context window) and a meter ticking on every word it processes. How should it find the answer?

The industry's default answer for the last few years has been *retrieval-augmented generation* — RAG. You break documents into chunks, store them in a vector database, and at question time you fetch the few chunks that look most similar to the question, then feed those chunks to the

model. RAG is a clever workaround for limited attention spans, and dozens of variants now exist.

But two things have changed recently. First, attention spans have ballooned: models now read a million tokens at once without breaking a sweat. Second, the cost per token has dropped enough that "just give it everything" is often cheaper than a developer's time spent tuning a clever pipeline. So the question of *when* RAG still earns its complexity, and *when* it has been overtaken by simpler approaches, deserves a careful empirical answer.

In plain English

We tested seven different ways of getting an AI model to answer questions from a corpus of documents, under identical conditions, and measured exactly how much each one costs, how often it gets the right answer, and how often it lies when the answer isn't there.

This paper provides such an answer for one well-controlled setting. We compare four RAG variants and three non-retrieval techniques under matched conditions: same documents, same questions, same model, same prompts (except where the technique demands otherwise). The results are not a universal verdict our corpus is small and our model is fixed but the seven head-to-head numbers in this paper are honest, reproducible, and inexpensive enough that the whole project ran on a personal API key for one dollar.

2 The seven approaches, in plain English

The seven approaches we compare fall into two families. RAG approaches *look up* relevant chunks before answering. Non-RAG approaches process the whole corpus directly. Figure 1 shows each one as a sequence of steps.

Naive RAG. The simplest RAG. Convert the question to a vector (an "embedding" a list of numbers that captures meaning), find the five chunks whose vectors are closest to the question's vector, and feed them to the model along with the question. One look-up, one answer.

Hybrid RAG. Combines two look-ups. Vector search (good at meaning) plus BM25, a classic keyword-search algorithm (good at literal phrasing). The two ranked lists are merged using a method called Reciprocal Rank Fusion, then the top five fused chunks go to the model.

HyDE (Hypothetical Document Embeddings). A counter-intuitive trick: ask the model to *invent* an answer first a confident-sounding made-up paragraph and search for chunks that look like that invented answer. The intuition is that an invented answer is closer in shape to a real answer than the question is. Costs two model calls per question.

Reranker RAG. Pull a wider net first (twenty candidate chunks instead of five), then use a smaller, dedicated "cross-encoder" model to rescore each candidate against the question. Take the top five after rescoring. The rescoring step costs almost nothing the cross-encoder runs on your laptop.

Full-Context Injection. No retrieval at all. Concatenate the entire corpus and put it directly into the model's prompt. Works only as long as the corpus fits in the context window. Cost scales linearly with corpus size.

Recursive Summarization. A classic map-reduce. First, summarize each document (telling the summarizer to preserve specific facts). Then concatenate the summaries and answer from those. Cheap per question after the one-time summary work.

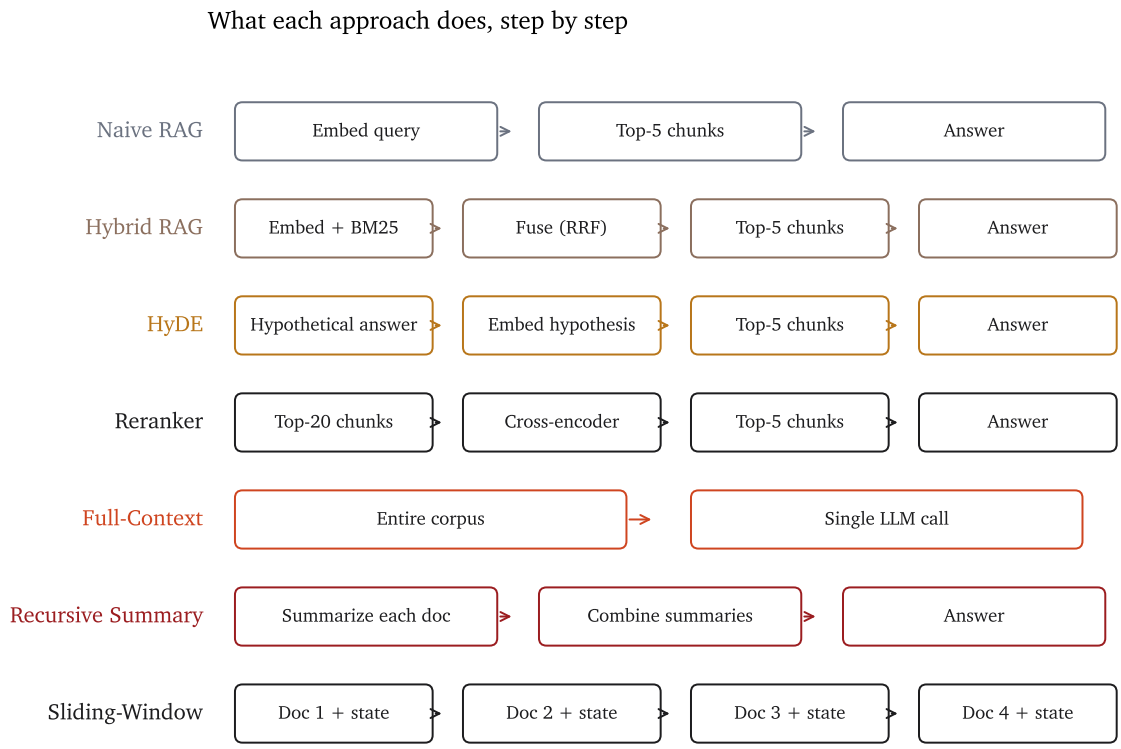


Figure 1: *The seven approaches at a glance. Each row is a complete pipeline from question to answer; each rounded box is one step the model takes.*

Sliding-Window Reasoning. Process the corpus one document at a time, while carrying forward a running "best answer" from one document to the next. Four documents means four model calls per question, each one revising the answer with new information.

3 How we tested them

3.1 The corpus: small but deliberate

We wrote our own four-document corpus rather than using a public benchmark. Each document is about 1,500–2,000 words and describes a fictional institution, city, satellite, or treaty. The documents are interconnected: the satellite mentioned in one document was built by the institute mentioned in another, which is located in the city in a third document, and partly funded by the treaty in the fourth. This intentional cross-document structure lets us probe "multi-hop" reasoning questions whose answers require linking facts across documents.

Why authored, not downloaded? Three reasons. First, we know every gold answer cold we wrote them. Second, we can plant specific identifiers (a password `MULBERRY-7Q`, a frequency `4.832 GHz`) at known depths to test "needle-in-haystack" retrieval. Third, we can ensure that some questions have answers that simply do not exist in the corpus, to test whether the model honestly refuses or fabricates.

3.2 The questions: five probes of capability

Our 34-question evaluation set spans five categories of difficulty:

- **Sanity (4 Q)** trivial single-fact questions. "In what year was the Halden Institute founded?" If an approach can't get these right, something is broken.
- **Needle-in-haystack (8 Q)** specific identifiers planted in the prose. "What is the satel-

Approach	Family	EM (mean ± std)	F1 (mean ± std)	Cost (USD)
Full-Context	non-RAG	0.667 ± 0.017	$\mathbf{0.881 \pm 0.012}$	0.0912
Sliding-Window	non-RAG	$\mathbf{0.696 \pm 0.045}$	0.875 ± 0.025	0.0972
HyDE	RAG	0.667 ± 0.017	0.838 ± 0.012	0.0186
Naive RAG	RAG	0.667 ± 0.017	0.836 ± 0.012	0.0122
Reranker	RAG	0.667 ± 0.017	0.833 ± 0.009	0.0123
Hybrid RAG	RAG	0.627 ± 0.017	0.797 ± 0.010	0.0122
Recursive Summary	non-RAG	0.206 ± 0.000	0.206 ± 0.000	0.0068

Table 1: *Headline results across all 7 approaches. Mean and standard deviation across 3 random seeds; cost is the average over 34 questions. Bold cells mark the best in each column.*

lite's internal designator?"

- **Multi-hop (8 Q)** answers require chaining facts across two or three documents. "In what city is the institute that built the FILAMENT-3 biosensor headquartered?"
- **Distractor (8 Q)** near-duplicate facts intentionally placed nearby. "In what year was MERIDIAN-7 launched (not MERIDIAN-6)?"
- **Hallucination probe (6 Q)** questions whose answers do not exist in the corpus. The correct response is "I don't know." Anything else counts as fabrication.

Every gold answer was verified by direct text search against the corpus, and every hallucination probe was verified to have no answer in the corpus.

3.3 The metrics, the model, the budget

We score answers two ways. **Exact Match (EM)** is 1 if the normalized prediction equals the normalized gold answer and 0 otherwise (lower-cased, articles stripped). **Token F1** gives partial credit for token overlap. For hallucination probes, a prediction counts as correct if it contains a refusal phrase like "not in context" or "I don't know."

In plain English

Exact Match is strict "Twenty-five years" is wrong when the gold is "25 years." **Token F1** forgives wordiness it gives partial credit for getting the right key word. Real-world "did the system actually answer correctly?" usually lies between these two.

A single model, `gemini-3-flash-preview`, handles every generation step for every approach. Embeddings come from the freely available `all-MiniLM-L6-v2` sentence transformer, and the reranker is the freely available `bge-reranker-base`. We set the model's temperature to 0.3 (mildly stochastic) and run each approach with three different random seeds to characterize seed-to-seed variance.

The total project budget was capped at $5 USD. A budget-aware wrapper around each API call refused to start any new call that would push cumulative spend above 95% of the cap. The full study spent $1.09 about 22% of the cap.

4 Results

4.1 The headline numbers

Table 1 reports the headline metrics for all seven approaches, sorted by token F1. Figure 2 shows the same numbers as a forest plot with bootstrap 95% confidence intervals.

4.2 Where each approach lives or dies

The headline F1 hides important variation across probe types. Figure 3 shows the per-probe-type breakdown. A clean visual story emerges:

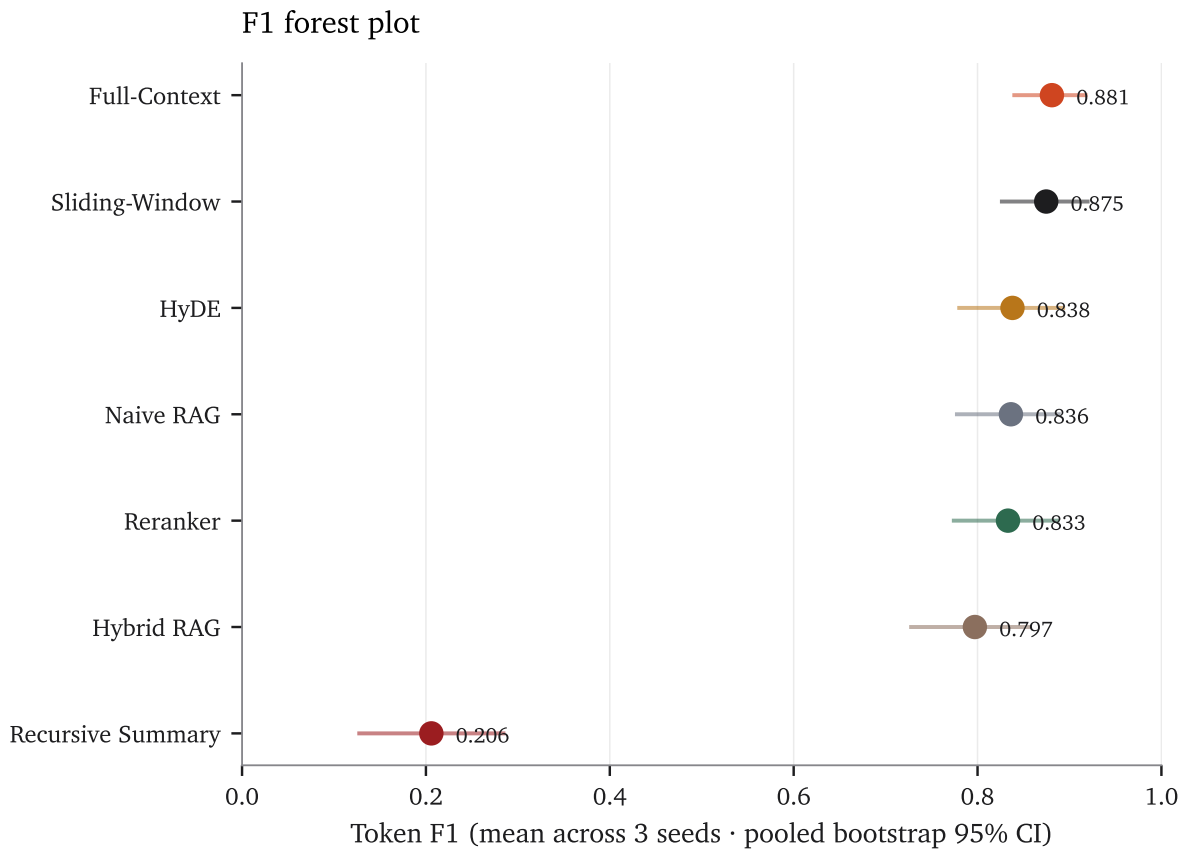


Figure 2: *Token F1 with pooled bootstrap 95% confidence intervals. The horizontal bars represent uncertainty; the dot is the mean across seeds. Recursive Summarization is clearly worse than every other approach; the top six are tightly clustered with overlapping intervals.*

- Every approach gets the **sanity** questions right.
- Every approach *except Recursive Summary* achieves perfect refusal on the **hallucination** probes.
- The **needle** questions are where the verbosity ceiling shows up most: all approaches retrieve the right fact, but a third of them wrap it in extra words and lose EM credit.
- The **multi-hop** and **distractor** probes are where approaches diverge most: full-context's wider view helps more than any retrieval trick.

4.3 Cost: where the dollars go

Figure 4 shows the cost per full eval-set run for each approach. The four single-LLM-call RAG variants all cluster around $0.012, an order of magnitude cheaper than Full-Context ($0.091) or Sliding-Window ($0.097). Recursive Summary is the cheapest at $0.007, but the savings are illusory its answer quality is unusable.

4.4 The cost-quality trade-off, on one chart

Figure 5 arranges every approach on a single map: cost on the horizontal axis, quality on the vertical, with the quadrant labels written out plainly. The story is at-a-glance:

- The two **premium** non-retrieval approaches (Full-Context and Sliding-Window) sit in the top-right: high quality, high cost.
- The four cheap-class RAG variants cluster tightly around $0.012, with HyDE, Naive, and Reranker statistically tied at F1 $\approx$ 0.83 and Hybrid slightly behind at 0.80. Reranker is

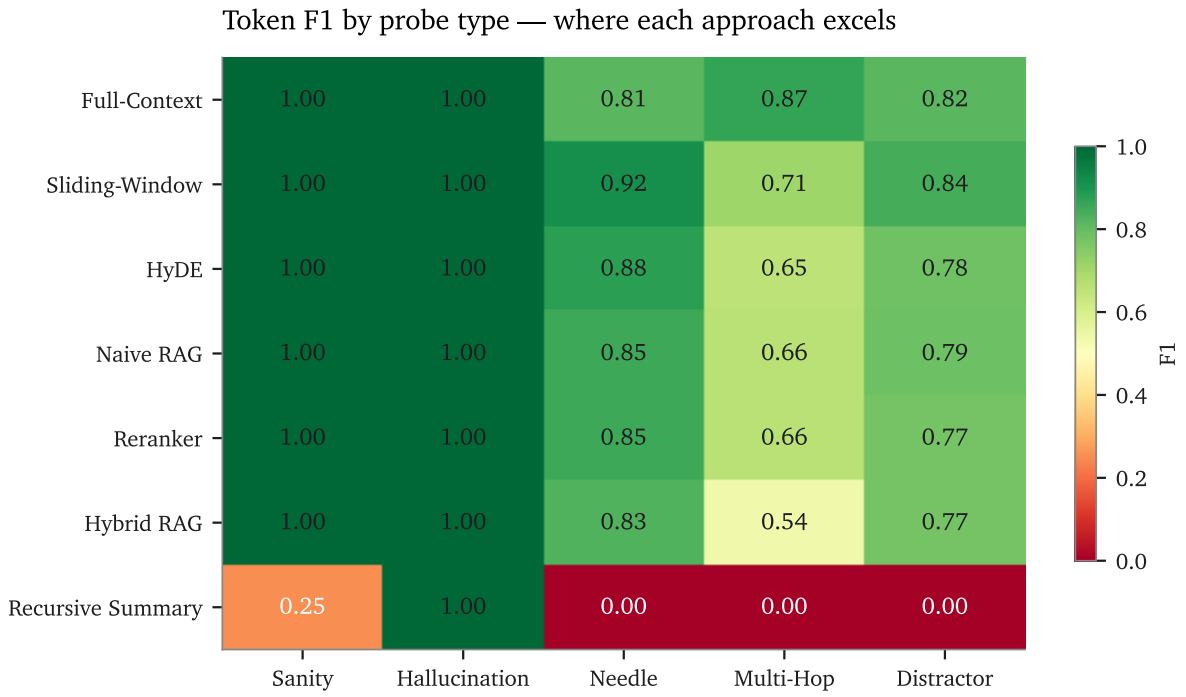


Figure 3: *Token F1 by probe type. Greener cells are better. Recursive Summary is the only approach with red on most columns; the other six are uniformly green except for the multi-hop and distractor columns where small differences appear.*

the **best value** of the four same dollar cost, highest F1 with the tightest variance.

- Recursive Summary is cheapest of all but anchors the bottom of the F1 axis its savings are meaningless because the answers are wrong.
- No approach occupies the bottom-right "dominated" quadrant (expensive AND low-quality), which means none of the seven is strictly worse than another on both axes simultaneously.

4.5 The trust map: honesty vs capability

Perhaps the most interesting result for production use is Figure 6. We plot each approach on two axes: factoid accuracy (how often it correctly answers questions whose answer exists) and refusal accuracy (how often it correctly says "I don't know" when it shouldn't know). Six of seven approaches sit firmly in the top-right corner both honest AND capable. Only Recursive Summary slips down on the capability axis, while remaining honest by accident (it refuses everything because it has lost the facts).

4.6 How reproducible are the results?

We ran each approach three times with different random seeds. Figure 7 shows how much F1 varies between runs. Five of seven approaches show essentially no variance (standard deviation ≤ 0.012). **Sliding-Window is the exception**, with about $3\times$ the variance of the others its four sequential model calls let small differences compound.

5 Five things we learned

5.1 Finding 1 Non-retrieval wins when the corpus fits

Full-Context and Sliding-Window hold the top two F1 spots (0.881 and 0.875). Their advantage is structural: when every fact is visible in the prompt, the model has nothing to mis-retrieve. RAG approaches are essentially making a one-shot bet that the right chunks landed in the top-5;

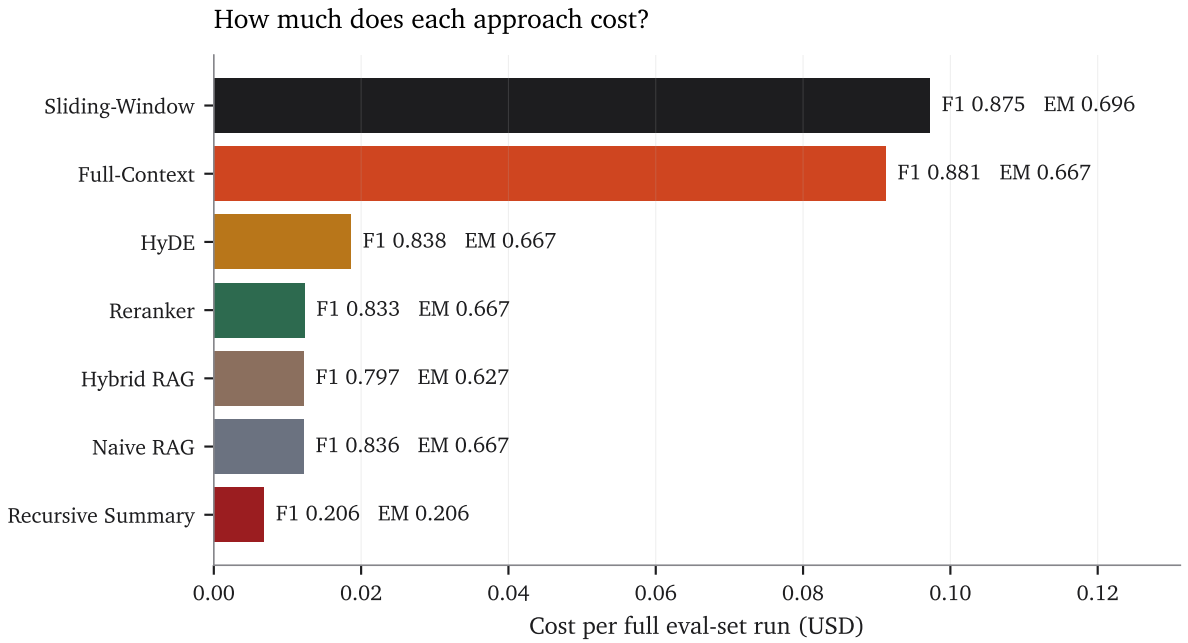


Figure 4: *Cost per full eval-set run, with EM and F1 annotated beside each bar. The four RAG variants are nearly identical in cost; the two large-context approaches cost about 7× more.*

non-retrieval approaches don't make that bet.

Key takeaway

When your documents fit in the context window, RAG's chunk-selection step is overhead, not optimization. Consider hand-curated retrieval only when the corpus is too large to fit, or when retrieval cost is dominated by storage rather than compute.

5.2 Finding 2 Pre-summarization is a quiet disaster

Recursive Summarization scored F1 = 0.206 about a quarter of the other approaches. Our summarization prompt explicitly told the model to "preserve ALL specific facts: names, dates, numbers, identifiers, percentages." It did not. Specific identifiers like `MULBERRY-7Q`, `M7-RX-204-A`, and `12%` vanished from the summaries, as did even mundane facts like a satellite launch date ("9 April 2019"). Only facts mentioned multiple times across the corpus survived.

Key takeaway

If your questions are about specific facts, never summarize first. Summarization is a useful compression technique for narrative content, but it is fundamentally incompatible with factoid retrieval no matter how strict your "preserve facts" prompt is.

5.3 Finding 3 Honesty is universal (almost)

The strict-refusal prompt "if the context does not contain enough information, respond with NOT_IN_CONTEXT" transferred cleanly across all six retrieval-style approaches. Every one of them refused correctly on all six hallucination probes. This is the single most important property to preserve in production, and the result is encouraging: the prompt does not depend on the retrieval strategy.

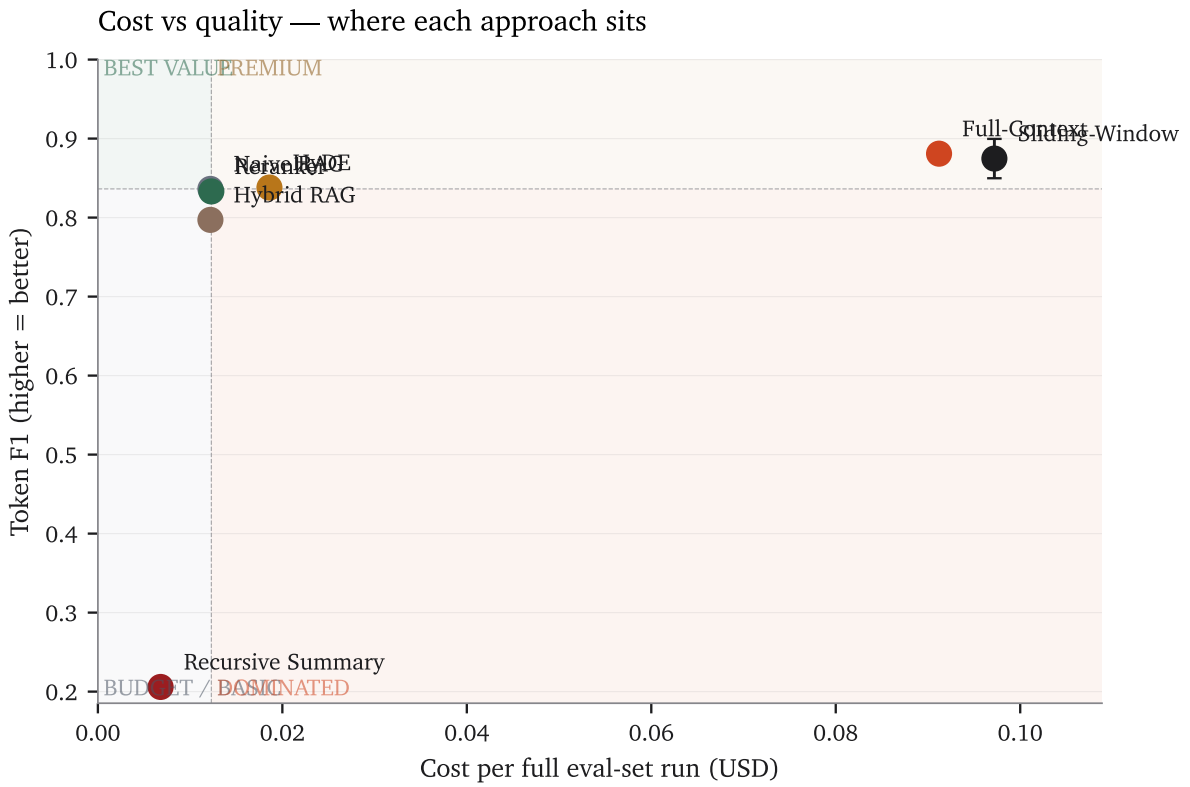


Figure 5: *Cost-quality quadrant map. Dashed lines mark the median cost and the median F1, splitting the plane into four labeled regions.*

Key takeaway

A well-designed refusal prompt is robust. If you give the model permission to say "I don't know," it usually will across very different pipelines.

5.4 Finding 4 Exact Match underestimates the model

Six of seven approaches cluster at EM ≈ 0.65, despite their F1 scores spanning 0.797 to 0.881. The reason is straightforward: the model often produces verbose-but-correct answers ("Twenty-five years" instead of "25 years"; "Dr. Mei-Ling Forrester" instead of "Mei-Ling Forrester"). EM penalizes these as wrong; F1 gives partial credit.

Key takeaway

For short factoid QA, report F1 alongside EM. EM alone systematically underestimates correctly-functioning systems by tens of percentage points.

5.5 Finding 5 Multi-step approaches need multi-seed evaluation

Sliding-Window's F1 standard deviation was 0.025, three times that of single-call approaches. Its first random seed scored 0.896 the highest of any run in the study. Its three-seed mean fell to 0.875. Without the multi-seed sweep we would have over-claimed sliding-window as a clear winner; with three seeds, it sits second behind Full-Context.

Key takeaway

The more steps your pipeline has, the more reproducibility you should demand. Single-shot benchmarks of multi-step pipelines can mislead by 5 percentage points easily.

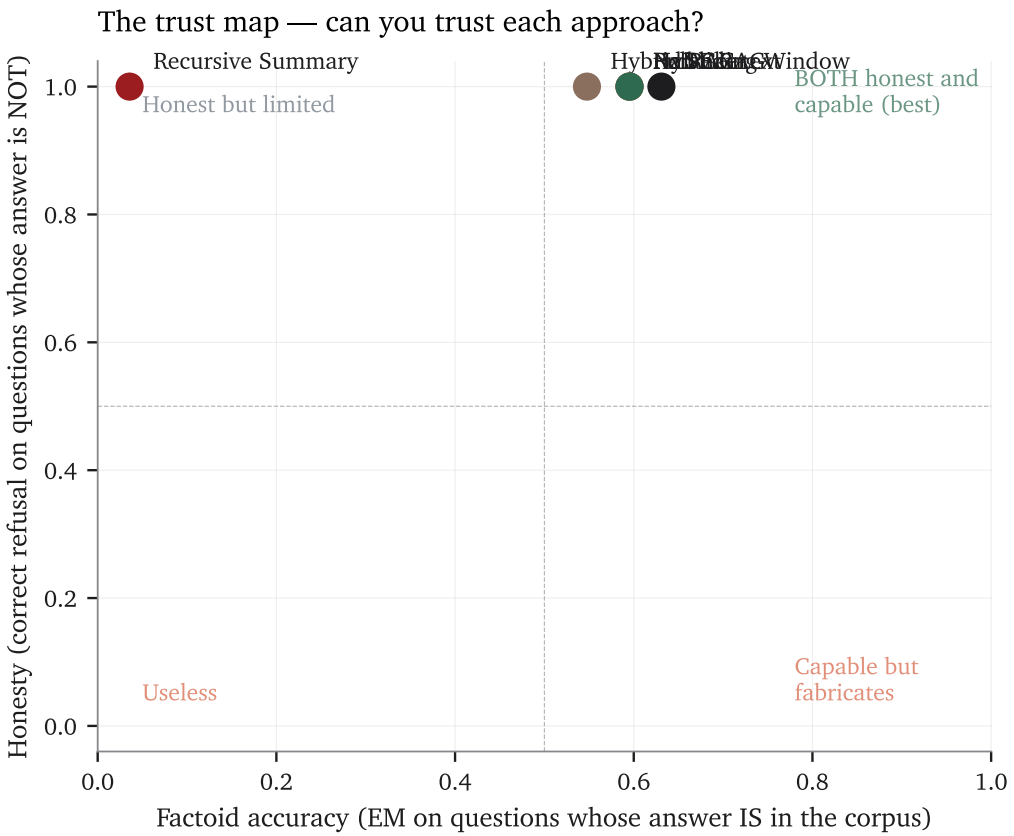


Figure 6: *The trust map. Six of seven approaches occupy the top-right "honest and capable" quadrant. No approach is in the dangerous bottom-right (capable but fabricating).*

6 When should you use which?

For a practitioner who has read this far, Table 2 is the executive summary. Find the row that matches your priority; the recommended approach is on the right.

7 Limitations and what we did not test

We tested one corpus, one model, one prompt style, one budget. Several axes are out of scope for this paper but worth naming clearly so a reader can judge generality.

Corpus size. Our corpus is about 8,000 words comfortably inside any modern context window. The picture changes once the corpus is too large to fit. We expect RAG approaches to start winning when full-context becomes prohibitive (either by cost or context-limit). Where exactly that crossover happens at 50,000 words? 500,000? is the obvious follow-up experiment.

Model. Results are reported only for `gemini-3-flash-preview`. A different model particularly one with weaker long-context attention might tell a different story, especially for Sliding-Window where state must be faithfully carried forward.

Probe coverage. 34 questions across 5 probe types is small. We chose breadth over depth to test diverse capabilities; the trade-off is that statistical claims about subtle differences between approaches (say, Reranker vs Naive RAG at F1 0.833 vs 0.836) are not defensible at this sample size. We would need on the order of 200–500 questions per probe type to separate the top six approaches with confidence.

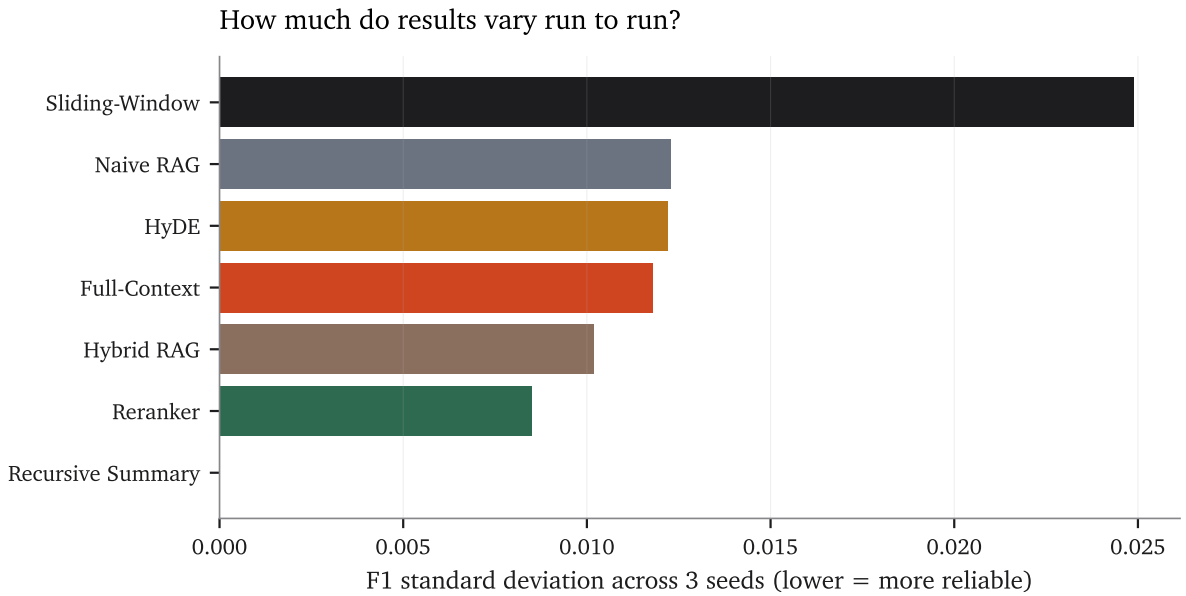


Figure 7: *F1 standard deviation across three random seeds. Lower means more reproducible. Multi-step pipelines like Sliding-Window inherently vary more than single-call approaches.*

Your priority	What that looks like in practice	Use
Cheapest	Cost is the binding constraint; quality ≥ 0.80 acceptable.	**Reranker**
Best quality	Whatever it costs, give me the highest F1. Corpus fits in context.	**Full-Context**
Auditable steps	You want to see what the model was thinking, document by document.	**Sliding-Window**
Multi-hop heavy	Most questions require chaining facts across documents.	**Full-Context**
Honesty critical	Wrong answer worse than no answer.	**Any of top 6**
Tight context window	Documents already too large to fit comfortably.	**Reranker**
Avoid at all costs	Factoid QA that needs specific identifiers, dates, numbers.	**Recursive Summary**

Table 2: *Decision chart. Choose the row whose "priority" matches yours; the rightmost column is the approach we recommend based on our results.*

Other techniques. We dropped 14 retrieval techniques and 5 non-retrieval techniques from this study to keep it tractable: GraphRAG, RAPTOR, MemoRAG, Self-RAG, Agentic RAG, Adaptive RAG, Parent-Child Retrieval, Multi-Query Retrieval, Hierarchical RAG, Contextual Compression, prompt compression (LLMLingua-2), explicit context caching, MemGPT-style memory, multi-agent memory, and several others. They become natural future-work candidates.

8 Reproducibility

Every result in this paper can be regenerated with two commands:

```
make install
make ui   # rebuilds the dashboard with the latest run data
```

The complete artifact set includes: the 34 authored questions and 4 authored corpus documents; the Python implementations of all 7 approaches under `src/ragureap/approaches/`; one run-folder per approach per seed under `runs/`, each containing the per-question prediction, the gold answer, the model's per-step trace with token counts and dollar cost; the multi-

seed aggregator and the figure-generation scripts under `scripts/`; this paper's source under `paper/draft.tex` and a one-line build command using `tectonic`; and a Next.js dashboard for interactive exploration of all of the above.

Total cost to reproduce the study, end to end, on a personal Gemini API key: about $1.10.

9 Conclusion

A surprisingly simple picture emerges from this study. **When the documents fit in the prompt, give them all to the model.** When they don't, use the cheapest retrieval method that does the job well — in our results, the cross-encoder reranker — and avoid the temptation to "compress" the corpus first via summarization, because you will reliably lose the facts you came to retrieve. The strict-refusal prompt earns its keep across every retrieval strategy, suggesting that honesty is largely a matter of asking nicely.

The most pragmatic recommendation from this work is not to commit to a single approach but to keep two in your toolkit: a small reranker-based RAG for everyday use, and a full-context fallback for when the question is hard enough or the corpus small enough that the extra dollar buys real F1. Everything else is interesting, but optional.

In plain English

The simplest summary: **when in doubt, hand the model the whole document**. When you can't, look up a few good chunks and ask carefully. Don't summarize the document first — you'll lose what you came for. And always give the model permission to say "I don't know."